République algérienne démocratique et populaire
Université des Sciences et de la Technologie Houari Boumediene
Faculté d'informatique

Rappels sur les bases de l'Algorithmique

Algorithmique et Structure de données (L2 - Informatique)

Définition d'un algorithme

Un algorithme est **un ensemble d'instructions bien définies** pour **résoudre** un problème particulier. Il prend un **ensemble d'entrée(s)** et **produit** la **sortie** souhaitée .(Encyclopédie Larousse).

Un algorithme est une procédure de **calcul bien définie** qui prend en **entrée une valeur** ou un ensemble de valeurs et **produit en sortie une valeur** ou un ensemble de valeurs. Un algorithme est donc une **séquence d'étapes de calcul** qui transforment l'entrée en sortie.(Cormen, Leiserson, Rivert).

Un algorithme est **un ensemble d'opérations** de calcul **élémentaires** organisées selon des **règles précises** afin de **résoudre** un problème donné. Pour chaque donnée du problème, il renvoie une solution après **un nombre fini** d'opérations.(Beauquier, Berstel, Chrétienne)

Un algorithme est une séquence **finie** et **non ambiguë** d'opérations ou d'instructions pour **résoudre** une **classe de problèmes**

Propriétés d'un algorithme

1- Validité: La validité d'un algorithme réside dans sa capacité à effectuer exactement la tâche pour laquelle il a été conçu.

2- Robustesse : L'algorithme robuste est conçu pour fonctionner de manière fiable et précise dans diverses conditions, y compris des circonstances inattendues ou défavorables. (prise en compte des différentes instances, cas particuliers, etc.)

3- Réutilisabilité : La réutilisabilité d'un algorithme est sa capacité à être réutilisé dans différents contextes ou pour résoudre plusieurs problèmes connexes sans modification significative.

4- Complexité: La complexité d'un algorithme est la quantité de ressources nécessaires pour traiter les entrées. Les principales ressources mesurées sont le temps (nombre d'instructions utilisées) et l'espace (quantité d'espace mémoire nécessaire).

Algorithmique et Structures de Données

Algorithmique

L'algorithmique est l'étude systématique de la conception et de l'analyse des algorithmes. L'algorithmique s'intéresse à l'art de construire des algorithmes et d'évaluer leurs propriétés (validité, robustesse, réutilisabilité, complexité et efficacité). L'algorithmique permet de passer d'un problème à résoudre à l'approche de résolution de problèmes (Algorithme).

L'approche de la résolution de problèmes

Analyser le problème: obtenir une description du problème.

Définir les données: leurs caractéristiques, leurs types.

Définir les résultats: leurs caractéristiques, leurs types.

Définir la relation entre les résultats et les données: Comment passer des données aux résultats ?

Structures de Données

Une structure des données est un moyen de stocker et d'organiser les données afin qu'elles puissent être utilisées efficacement pour le stockage, le traitement et la récupération. Une structure de données est une implémentation concrète d'un type de données abstrait .

"Je vais en fait affirmer que la différence entre un mauvais programmeur et un bon est de savoir s'il considère son code ou ses structures de données plus importantes.
Les mauvais programmeurs s'inquiètent du code. Les bons programmeurs se soucient des structures de données et de leurs relations. "

— Linus Torvalds (creator of Linux)



Structure générale d'un algorithme

Partie d'en-tête

Partie déclaration

Partie action

```
Algorithm <nameAlgo> ;

Const <IdObj> = <ValueObj> ;
Type <IdObj> = <idType> ;
Var <IdObj> : <TypeObj> ;

Begin
    <Action1> ;
    <Action2> ;
        ---
    <ActionN> ;
End.
```

Actions

L'affectation :

<IdObj> ← **<Expression> ;**

Structures de contrôle :

Instructions de sélection (instructions conditionnelles)

If <Condition> **Then** <Block Action> **Endif;**

If <Condition> **Then** <Block Action1> **Else** <Block Action2> **Endif;**

Structure à conditions multiples (instructions Switch)

Case <Expression> **Of**
<Val1> **:** <Block Action1>;
<Val2> **:** <Block Action2>;

<ValN> **:** <Block ActionN>;
Else <Block OtherActions> ;
EndCase;

Structures itératives (boucles) :

While <Condition> **Do** <Block Action> **Endwhile;**

Repeat <Block Action> **Until** <Condition> **;**

For <IdCompt> ← <IV> **To** <FV> **Do** <Block Action> **Endfor;**

For <IdCompt> **From** <IV> **To** <FV> **Do** <Block Action> **Endfor;**

Actions paramétrées (sous-routines "programmes") :

Fonctions :

Function <FunctionName> (<formal parameters >) : <TypeResult>**;**
Var < Local variables declaration > **;**
Begin
    <Block Action> **;**
    <FunctionName> ← <Expression> **;**
End;

```
Function <FunctionName> (<formal parameters >) : <TypeResult>;
Var < Local variables declaration > ;
Begin
        <Block Action> ;
         return <Expression> ;
 End;
```

Procédures:

```
Procedure <ProcedureName> (< formal parameters >) ;
Var < Local variables declaration > ;
Begin
        <Block Action> ;
End;
```

Appel :

<FonctionName> (<Effective parameters>) **Agit comme** : **Expression**

<ProcedureName> (< Effective parameters>) **Agit comme** : **Action**

Modes de passage des paramètres :

Passage par valeur: Dans ce cas, le sous-programme utilise des **copies** des données passées en paramètres. Une fois terminé, les valeurs d'origine restent inchangées.

Les paramètres effectifs sont gérés comme des variables locales.

Passage par référence (adresse) : Dans ce cas, le sous-programme utilise les données d'origine. Une fois l'opération terminée, les variables d'origine sont affectées par toutes les modifications.

Les paramètres effectifs sont gérés comme des variables globales.

Structures de Données

Types de base (simple) : entier, réel, caractère, booléen

Tableau : regroupement de variables de **même type** de données.

Enregistrement : regroupement de variables de **différents types** de données

Fichier : représentation des données dans la mémoire secondaire.

Liste : allocation dynamique de mémoire (série de nœuds chainée).

Programme

1- Analyse des Algorithmes (Etude de la complexité)

2- Allocation Dynamique et Liste Chaînée (LSC, LDC, LCC)

3- Les Piles et Les Files

4- La Récursivité

5- Les Arbres

6- Tables de hachage

Algorithmique et Structures de données

1 - 4

2 - 3 - 5 - 6